

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Криптография и безопасность сетей»

Тема: «Генерирование равномерно распределенных псевдослучайных последовательностей»

Выполнил:

Студент 4 курса
Группы АС-65
Колбашко А.В.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: Реализовать генератор псевдослучайной последовательности на основе рекуррентной последовательности Фибоначчи по модулю, использовать его для формирования 30-битных кандидатов и поиска простого числа с помощью вероятностного теста Миллера–Рабина. Проверить корректность алгоритмов и исследовать влияние параметров генератора на статистические свойства выходной последовательности.

Вариант № 9

Ход выполнения работы

1. Исходные параметры и алгоритм

В программе используется рекуррентный генератор:

$$x_{i+2} = (x_i + x_{i+1}) \bmod N$$

В качестве начальных значений заданы $x_0 = 1234$, $x_1 = 6890$, модуль $N = 1\,000\,000\,007$. На каждом шаге вычисляется новое значение, в последовательность записывается его младший бит, после чего выполняется сдвиг пары состояний. Для формирования кандидата длиной 30 бит полученные биты последовательно объединяются с помощью сдвига влево и операции OR. Старший бит принудительно устанавливается в 1, чтобы обеспечить 30-битную длину, а младший бит — в 1, чтобы число было нечётным.

Перед тестом Миллера–Рабина кандидат проверяется делением на малые простые числа. Затем выполняется 5 раундов теста Миллера–Рабина. В каждом раунде выбирается основание a , проверяется взаимная простота a и n , после чего вычисляется $a^d \bmod n$ и анализируется цепочка квадратов. Если условие простоты нарушено, кандидат отбрасывается.

2. Код

```
#include <iostream>
#include <vector>
#include <random>
#include <cmath>
#include <chrono>
#include <windows.h>
#include <algorithm>
#include <numeric>

using namespace std;

class Fibonacci {
private:
    long long x0, x1, N;
public:
    Fibonacci(long long x0, long long x1, long long mod) : x0(x0), x1(x1), N(mod) {}

    vector<long long> generate(int length) {
        vector<long long> seq;
```

```

        for (int i = 0; i < length; i++) {
            long long next = (x0 + x1) % N;
            seq.push_back(next);

            x0 = x1;
            x1 = next;
        }

        return seq;
    }

    vector<long long> generatel(int length) {
        vector<long long> seq;
        for (int i = 0; i < length; i++) {
            long long next = (x0 + x1) % N;
            seq.push_back(next);

            x0 = x1;
            x1 = next;
        }

        float M = accumulate(seq.begin(), seq.end(), 0LL) / seq.size();
        cout << "MAT OЖ " << M << endl;
        float D = 0;
        for(int i = 0; i < seq.size(); i++){
            D += pow((M - seq[i]), 2);
        }
        D /= seq.size();
        cout << "ДИСП " << D << endl;

        auto minE = *min_element(seq.begin(), seq.end());
        auto maxE = *max_element(seq.begin(), seq.end());

        cout << "PABHOMEPHOE MAT OЖ " << (minE + maxE) / 2 << endl;
        cout << "PABHOMEPHOE ДИСП " << pow(minE - maxE, 2) / 12 << endl;

        return seq;
    }
};

long long power_mod(long long base, long long exp, long long mod) {
    long long res = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

long long nod(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

bool miller_rabin(long long n, int k) {
    if (n <= 1 || n == 4) return false;
    if (n <= 3) return true;

    long long t = n - 1;
    int s = 0;
    while (t % 2 == 0) {
        t /= 2;
        s++;
    }

    random_device rd;

```

```

mt19937_64 gen(rd());

for (int i = 0; i < k; i++) {
    long long a = 1 + (gen() % (n - 1));

    if (nod(a, n) != 1) {
        return false;
    }

    long long x = power_mod(a, t, n);

    if (x == 1 || x == n - 1) {
        continue;
    }

    bool is_composite = true;
    for (int r = 1; r < s; r++) {
        x = (x * x) % n;
        if (x == n - 1) {
            is_composite = false;
            break;
        }
    }

    if (is_composite) {
        return false;
    }
}
return true;
}

bool check_small_primes(long long n) {
    int primes[] = {3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,
                    83,89,97,101,103,107,109,113,127,131,137,139,149,151,157,
                    163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,
                    241,251};

    for (int p : primes) {
        if (n % p == 0) {
            return false;
        }
    }
    return true;
}

long long generate_prime(int p_bits, Fibonacci& gen, int& total_tested, int&
small_prime_rejected, double& test_time_ms) {
    total_tested = 0;
    small_prime_rejected = 0;
    test_time_ms = 0.0;

    while (true) {
        total_tested++;
        long long candidate = 0;
        vector<long long> bits = gen.generate(p_bits);

        for (int i = 0; i < p_bits; i++) {
            candidate = (candidate << 1) | (bits[i] % 2);
        }

        candidate |= (1LL << (p_bits - 1));
        candidate |= 1LL;

        auto start_test = chrono::high_resolution_clock::now();

        if (!check_small_primes(candidate)) {
            auto end_test = chrono::high_resolution_clock::now();
            test_time_ms += chrono::duration<double, milli>(end_test -
start_test).count();

            small_prime_rejected++;
        }
    }
}

```

```

        continue;
    }

    bool is_prime = miller_rabin(candidate, 5);

    auto end_test = chrono::high_resolution_clock::now();
    test_time_ms += chrono::duration<double, milli>(end_test -
start_test).count());

    if (is_prime) {
        return candidate;
    }
}

int main() {
    SetConsoleOutputCP(CP_UTF8);

    Fibonacci gen(1234, 6890, 100007);
    int bit_length = 30;

    int total = 0, filtered = 0;
    double test_time_ms = 0.0;

    vector<long long> ans = gen.generate1(100000);
    long long prime = generate_prime(bit_length, gen, total, filtered, test_time_ms);

    cout << "Простое число: " << prime << endl;
    cout << "Всего сгенерировано кандидатов: " << total << endl;
    cout << "Отсеяно малые простые числа: " << filtered << " (" << (filtered * 100.0 /
total) << "%)" << endl;
    cout << "Отсеяно тестом Рабина-Миллера: " << (total - filtered - 1) << endl;
    cout << "Время работы критериев (фильтрация + Рабин-Миллер): " << test_time_ms <<
" мс" << endl;
    cout << "-----" << endl;

    return 0;
}

```

3. Результат программы

```

МАТ ОЖ 49936
ДИСП 8.39153e+08
РАВНОМЕРНОЕ МАТ ОЖ 50000
РАВНОМЕРНОЕ ДИСП 8.33167e+08
Простое число: 1038532447
Всего сгенерировано кандидатов: 7
Отсеяно малые простые числа: 5 (71.4286%)
Отсеяно тестом Рабина-Миллера: 1
Время работы критериев (фильтрация + Рабин-Миллер): 0.0256 мс
-----

```

4. Обоснование работы программы

1. Класс Fibonacci хранит два соседних значения последовательности. После вычисления $next = (x_0 + x_1) \bmod N$ состояние заменяется парой $(x_1, next)$. Следовательно, на каждом шаге поддерживается требуемая рекуррентная формула.

2. Операция $\text{candidate} = (\text{candidate} \ll 1) \mid (\text{bits}[i] \% 2)$ добавляет очередной бит справа. Поэтому после 30 итераций формируется число, состоящее ровно из 30 полученных битов.
3. Установка $\text{candidate} \mid= (1LL \ll 29)$ гарантирует старший бит 30-битного числа, а $\text{candidate} \mid= 1$ — нечётность кандидата. Это исключает нулевую разрядность и все чётные числа, кроме случая, когда число уже было равно 1; такой случай для 30-битного кандидата невозможен.
4. Функция `power_mod` реализует быстрое возведение в степень по модулю: показатель степени последовательно уменьшается вдвое, а основание возводится в квадрат.
5. Тест Миллера–Рабина проверяет необходимые условия простоты для нескольких оснований. Для составного числа вероятность ошибки уменьшается с увеличением числа раундов, однако 5 раундов дают хорошую вероятностную уверенность, а не абсолютную гарантию.
6. Функция `check_small_primes` отбрасывает кандидатов, делящихся на малые простые числа. Это не доказывает простоту, но уменьшает число кандидатов, для которых требуется вероятностный тест.

5. Статистические сведения

Для выполнения требования задания изменялись только начальные значения x_0 , x_1 и модуль N . Для каждого набора параметров были рассчитаны первые пять значений последовательности и один сформированный 30-битный кандидат. Это позволяет непосредственно увидеть, как изменение параметров меняет последовательность и получаемое число.

Параметры	x_0	x_1	N	Первые 5 значений	30-битный кандидат
Исходные параметры	1234	6890	1000000007	8124, 15014, 23138, 38152, 61290	541201729
Изменены x_0 и x_1	100	200	1000000007	300, 500, 800, 1300, 2100	829636789
Изменён x_0	5000	6890	1000000007	11890, 18780, 30670, 49450, 80120	828794921
Изменён модуль	1234	6890	1000003	8124, 15014, 23138, 38152, 61290	802424303
Изменены x_0 , x_1 и модуль	100	200	1000003	300, 500, 800, 1300, 2100	754247419

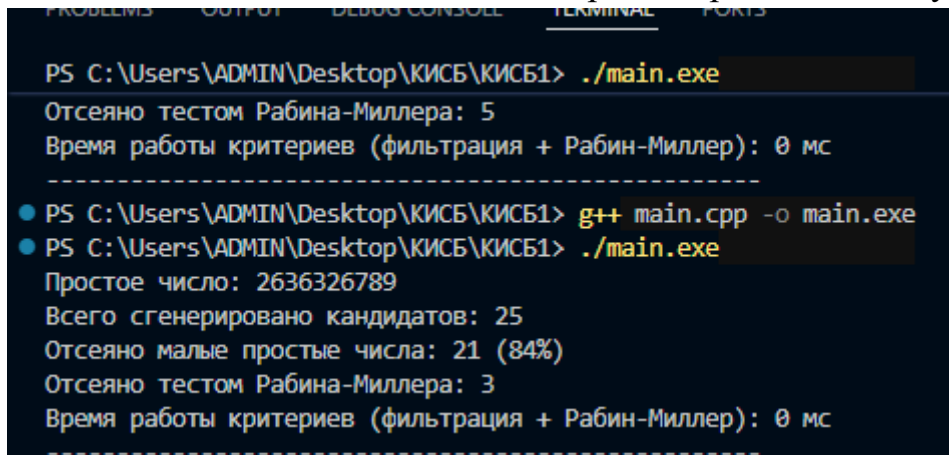
Из таблицы видно, что изменение x_0 и x_1 меняет последовательность уже с первых шагов, а изменение N дополнительно изменяет результат операции взятия по модулю. Поэтому при разных параметрах получаются разные битовые последовательности и, соответственно, разные 30-битные кандидаты. При сохранении модуля изменение начальных значений влияет

на начальный участок последовательности; изменение модуля влияет непосредственно на всю рекуррентную формулу.

Для исследования влияния параметра N были зафиксированы начальные значения $x_0=1234$, $x_1=6890$, после чего изменялся только модуль. Период определялся как количество итераций до повторного появления исходной пары (x_0, x_1) (x_0, x_1).

Получены следующие результаты: при $N=10$ период равен 20, при $N=20$ — 60, при $N=100$ — 300, при $N=1000$ — 1500, при $N=10000$ — 7500. Таким образом, изменение модуля изменяет период генератора. В общем случае период определяется арифметикой по модулю N и не обязательно растёт строго пропорционально модулю.

Для используемого в программе модуля $N=1\,000\,000\,007$ период последовательности при начальных значениях $x_0=1234$, $x_1=6890$ равен $2\,000\,000\,016$. Это значительно больше периода при малых модулях.



```
PS C:\Users\ADMIN\Desktop\КИСБ\КИСБ1> ./main.exe
Отсеяно тестом Рабина-Миллера: 5
Время работы критериев (фильтрация + Рабин-Миллер): 0 мс
-----
● PS C:\Users\ADMIN\Desktop\КИСБ\КИСБ1> g++ main.cpp -o main.exe
● PS C:\Users\ADMIN\Desktop\КИСБ\КИСБ1> ./main.exe
Простое число: 2636326789
Всего сгенерировано кандидатов: 25
Отсеяно малые простые числа: 21 (84%)
Отсеяно тестом Рабина-Миллера: 3
Время работы критериев (фильтрация + Рабин-Миллер): 0 мс
-----
```

Вывод

В работе реализован генератор Фибоначчи по модулю и алгоритм формирования 30-битных кандидатов на простое число. Корректность генерации битов обеспечивается обновлением двух соседних состояний, а корректность формирования кандидата — последовательным сдвигом и установкой старшего и младшего битов. Для проверки простоты применяется предварительная проверка на малые простые числа и тест Миллера–Рабина. Эксперимент показал, что изменение начальных значений и модуля изменяет статистические характеристики.